

How JavaScript code works Behind the Scenes >

```
var a = 10
```

```
var b = 20
```

```
function addNumber(num1, num2) {  
  var sum = num1 + num2;  
  return sum;  
}
```

①
Code run

```
var sumResult1 = addNumber(a, b);
```

```
var sumResult2 = addNumber(4, 5);
```

```
console.log(sumResult1, sumResult2);
```

code run ②

code run ③

Whenever you run the code → for that
Execution Context will be Created
and run on two Phases.

Memory
allocation
(memory
Allocate
hogi)

Execution
Phase

(code ko
execute phir
kiya jayega)

Code Run → Execution Context

① MA
Phase

② Exec
Phase

Phase 1 :

code ke andar jitne bhi variables hai unhe memory allocate karne hai bas...

①	②
memory alloc	exec
num1 = undefined	10
num2 = undefined	20
sum = undefined	30

addNumber(a, b)

Phase 2
num1 = 10

①	②
memory alloc	exec
num1 = undefined	10
num2 = undefined	20
sum = undefined	30

Global execution context

①	②
memory alloc	Execution mode
a = undefined	10
b = undefined	20
addNumber = { fn code }	

SumResult1 = undefined	30
SumResult2 = undefined	9

return sum

①	②
num1 = undefined	4
num2 = undefined	5
sum = undefined	9

addNumber(4, 5);

function ke andar wali code ko memory allocate abhi nahi hogi.

☆ Sab function call hogi tab hogi. [again] → Code run → Context

Phase 2 :
(Focus on code)
① code dobara read karega.
② ab code execute hoga...

so bhi instructions hogi usko execute krega [a = 10]

Phase 2 : EXEC Phase

Why undefined Appears Before Var Initialization

```
console.log(a);  
(your previous code here)
```

▷ code runned → Execution Context created
→ where memory allocation happens →
a = undefined, b = undefined, addNumbers fn code
→ sumResult1 = undefined, sumResult2 = undefined
→ then Execution Phase Start → read
code again from top → console.log(a) →
a = 10, b = 20 ... so on
↓
undefined

Let, Const vs Var

```
console.log(a);  
const b = 20  
let a = 10;
```

(X) cannot access 'a' before initialization

```
let a = 10  
const b = 20  
console.log(a);
```

// memory alloc

a = <uninitialized>
b = <uninitialized>

you can't access jab tak
value na chali jai
(Temporal dead zone)

// execution Phase

a = 10
b = 20

memory initialize hogyi
but access nahi kar sakte
jab tak value na chali
jai

↓
(30)